Sound of Silence

28th January 2024 / Document No. D24.102.191

Prepared By: `w3th4nds`

Challenge Author(s): `w3th4nds`

Difficulty: Easy

Classification: Official

Synopsis

- Sound of Silence is an easy difficulty challenge that features Buffer Overflow, calling the `gets` function to take as argument `system@PLT` and then enter there the string `bin0sh` to spawn shell. `ret2libc` or other techniques are not available because there is no function to print to `stdout`.

Description

- Navigate the shadows in a dimly lit room, silently evading detection as you strategize to outsmart your foes. Employ clever distractions to divert their attention, paving the way for your daring escape!

Skills Required

- Basic `ROP`.

Skills Learned

- Call `gets` with `system` as argument.

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
```

Protections 🛡️

As we can see:

Protection	Enabled	Usage
Canary	❌	Prevents **Buffer Overflows**
NX	✅	Disables **code execution** on stack
PIE	❌	Randomizes the **base address** of the binary
RelRO	**Full**	Makes some binary sections **read-only**

The program's interface

```
~The Sound of Silence is mesmerising~

>>
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaa
[1]    212434 segmentation fault (core dumped)  ./sound_of_silence
```

As we can see, the program is pretty straightforward. It asks for input and then it `SegFaults` after `40` bytes. This seems like a very easy challenge, but the tricky part is that there are no functions to print to `stdout` to leak any addresses and perform classic `ROP` techniques.

Disassembly

Starting with `main()`:

```
void main(void)

{
  char local_28 [32];

  system("clear && echo -n \'~The Sound of Silence is mesmerising~\n\n>> \'");
  gets(local_28);
  return;
}
```

The program is really simple. There is an obvious `Buffer Overflow` with `gets(local_28)`. The only other function we can use it `system`.

Exploitation Path

We will overwrite the `return address` with the address of `gets@PLT` and we will pass as argument the address of `system@PLT`. Then, whatever we enter, will be the argument of `system`. It's obvious that we will enter the string `/bin/sh` in there to spawn shell. The character `/` is converted so instead of this, we will use `0`. On the other hand, we can skip shell and just call `system("cat flag*");`. The same issues occurs so we use `cat glag*` instead.